

SA.4 — Seguridad y Protección de Datos

Evaluación 2 · Nivel SA

Mind Games — Ecosistema de Dispositivos Inteligentes

Juan Luis Mendoza Hernandez

Matrícula: 2023371106

Universidad Tecnológica de Querétaro
Desarrollo para Dispositivos Inteligentes

Cuatrimestre Mayo-Agosto 2026 · 07/08/2026

Índice

SA.4 — Seguridad y Protección de Datos Personales	3
1. Validación de event.origin en BroadcastChannel	3
El riesgo real (honesto, sin exagerar)	3
Qué ataque previene (el escenario donde Sí importa)	3
2. LFPDPPP — Datos personales tratados	4
3. La apiKey de Firebase — por qué no es un secreto (y por qué igual la sacamos del repo)	5
La apiKey web de Firebase es pública por diseño	5
La protección real son las Firestore Security Rules	5
Entonces, ¿por qué la mantenemos fuera del repositorio?	5
firebase.rules comentadas línea por línea	6
4. Aviso de privacidad completo	8
5. Plan de retención	8
6. Checklist de seguridad de la PWA	10

SA.4 — Seguridad y Protección de Datos Personales

Mind Games — Ecosistema de Dispositivos Inteligentes. Juan Luis Mendoza Hernandez · 2023371106

Este documento cubre los 6 puntos pedidos para SA.4. Todo el código citado es el código real del repositorio (rutas y números de línea verificables al momento de escribir esto); ningún fragmento es de relleno.

1. Validación de `event.origin` en `BroadcastChannel`

Ubicación real: `smart_pwa/js/broadcast.js`, función `escucharToggleSafeZone`.

```
// smart_pwa/js/broadcast.js, líneas 10–26
export function escucharToggleSafeZone(callback) {
  canal.onmessage = (evento) => {
    // BroadcastChannel ya restringe los mensajes al mismo origen por spec
    // – a diferencia de postMessage, ninguna pestaña de otro origen puede
    // unirse a este canal. Aun así validamos `origin` aquí por la misma
    // disciplina de defensa en profundidad que exige postMessage: si este
    // mismo patrón se reutiliza más adelante para comunicación cross-origin
    // (ej. un iframe embebido), la validación ya está puesta y evita que un
    // documento de otro origen inyecte mensajes falsos (spoofing) que la
    // TV trate como legítimos.
    if (evento.origin && evento.origin !== location.origin) return; // requisito SA.4

    if (evento.data && evento.data.tipo === 'toggle-safe-zone') {
      callback(evento.data.mostrar);
    }
  };
}
```

La comparación exacta está en la **línea 20**.

El riesgo real (honesto, sin exagerar)

`BroadcastChannel` con el mismo nombre de canal (`'mindgames'`, línea 4) **ya está restringido por especificación a páginas del mismo origen** — a diferencia de `window.postMessage`, no existe ningún mecanismo por el que una pestaña de `https://atacante.com` pueda unirse al canal `'mindgames'` abierto por `http://localhost:8080`. En ese sentido, para el uso actual (solo dos módulos de `smart_pwa` hablando entre pestañas de sí mismo) la validación es defensa en profundidad, no una mitigación de un ataque explotable hoy.

Qué ataque previene (el escenario donde SÍ importa)

El valor real de la validación aparece si este mismo patrón de código se reutiliza para un canal que **sí** puede cruzar orígenes — el caso típico es `window.addEventListener('message', ...)` para comunicarse con un `<iframe>` embebido (algo que este proyecto ya hace indirectamente: Firebase Auth abre un iframe hacia `mind-games-ddi.firebaseio.com`, documentado en la CSP de `index.html`). Sin la validación de origen, un documento controlado por un atacante podría:

1. Enviar un mensaje falso con `{ tipo: 'toggle-safe-zone', mostrar: true }` para manipular la UI (menor, en este caso).
2. En un canal con más superficie (ej. si en el futuro se usa para propagar `sessionState` entre pestañas sin pasar por Firestore), inyectar datos falsos que la TV trate como si vinieran de una sesión legítima — un *spoofing* de estado de aplicación.

La línea 20 es la guarda que, si el canal cambia de naturaleza más adelante, ya está puesta y no depende de que alguien se acuerde de agregarla bajo presión de tiempo.

2. LFPDPPP — Datos personales tratados

Ley Federal de Protección de Datos Personales en Posesión de los Particulares (México). Tabla de los datos reales que el ecosistema trata, con su finalidad y base legal:

Dato: Nombre	
Dónde vive (real)	<code>users/{uid}.nombre</code> (<code>auth_repository.dart:50</code>)
Finalidad	Identificar al usuario en la UI
Base legal (LFPDPPP)	Art. 8 — consentimiento del titular (tácito, al registrarse)
¿Sensible?	No

Dato: Correo electrónico	
Dónde vive (real)	<code>users/{uid}.email</code> + gestionado internamente por Firebase Auth
Finalidad	Autenticación, identificación de cuenta
Base legal (LFPDPPP)	Art. 8 — consentimiento del titular
¿Sensible?	No

Dato: Contraseña	
Dónde vive (real)	Nunca en nuestro Firestore — gestionada y hasheada íntegramente por Firebase Auth (Identity Platform)
Finalidad	Autenticación
Base legal (LFPDPPP)	Art. 8; además Firebase la trata bajo su propia política de seguridad (hash, nunca en texto plano, ni siquiera nosotros la vemos)
¿Sensible?	No (pero de alta criticidad)

Dato: Código de verificación (2FA)	
Dónde vive (real)	<code>users/{uid}/verificacion/actual.codigo</code> (<code>auth_repository.dart:89-106</code>), expira en 5 min
Finalidad	Segundo factor de autenticación
Base legal (LFPDPPP)	Art. 8 — consentimiento del titular
¿Sensible?	No

Dato: Ritmo cardiaco (<code>heartRate</code>)	
Dónde vive (real)	<code>sessionState/{uid}.heartRate</code> , <code>users/{uid}/sessions/{id}.heartRatePromedio</code>
Finalidad	Mostrar métricas de sesión, disparar alerta de umbral
Base legal (LFPDPPP)	Art. 8 + Art. 9 — consentimiento EXPRESO (dato de salud)
¿Sensible?	Sí — dato de salud

Dato: Tiempo de sesión, movimientos, concentración	
Dónde vive (real)	<code>sessionState/{uid}</code> , <code>users/{uid}/sessions/{id}</code>
Finalidad	Estadísticas de uso, panel "Sesión en curso"
Base legal (LFPDPPP)	Art. 8 — consentimiento del titular
¿Sensible?	No

Nota importante sobre el ritmo cardiaco: aunque en este proyecto el valor lo genera `SensorSimulator` (wearable, dato sintético, no un sensor real de hardware), el **campo en sí** es un dato de salud por naturaleza. La LFPDPPP no distingue "dato real" de "dato simulado" — clasifica por el TIPO de información que el campo representa. Por eso el ecosistema lo trata como sensible: es la única razón detrás de exigir consentimiento expreso (Art. 9) en el aviso de privacidad (sección 4) para ese campo en particular, y no solo el consentimiento tácito que basta para nombre/correo/tiempos.

3. La `apiKey` de Firebase — por qué no es un secreto (y por qué igual la sacamos del repo)

Este es un punto que se malinterpreta seguido, así que vale la pena ser preciso:

La `apiKey` web de Firebase es pública por diseño

`smart_pwa/js/firebase-config.js` y el bloque `web` de `telefono_app/lib/firebase_options.dart` contienen una `apiKey`. Esa llave **viaja dentro del bundle que se le sirve al navegador/dispositivo** — cualquiera que abra DevTools → Network o descompile el APK puede leerla. Google lo dice explícitamente en su propia documentación: la `apiKey` de un proyecto Firebase identifica al proyecto ante la API de Google, **no autentica ni autoriza nada por sí sola**. No es equivalente a una API key de, por ejemplo, un proveedor de pagos.

La protección real son las Firestore Security Rules

Lo que de verdad decide quién puede leer o escribir qué es `firestore.rules` (ver sección siguiente, comentada línea por línea). Aunque alguien tuviera la `apiKey` de este proyecto, sin una sesión de Firebase Auth válida las reglas le niegan todo acceso a `users/{uid}`, `sessionState/{uid}` y a escribir en `games`.

Entonces, ¿por qué la mantenemos fuera del repositorio?

Tres razones, ninguna es "porque es secreta":

1. **Buena práctica general:** aunque no sea explotable por sí sola, publicarla en un repo público facilita reconocimiento del proyecto/fingerprinting y simplifica ataques de fuerza bruta o abuso de cuota si se combinan con OTRAS fallas (ej. reglas mal escritas en el futuro).
2. **Requisito explícito de esta evaluación** (CLAUDE.md, regla 1): nunca commitear `firebase_options.dart`, `google-services.json`, `js/firebase-config.js` — siempre archivo real + plantilla `.example`.
3. **Rotación más simple:** si el proyecto cambia de dueño o se reconfigura, no hay que reescribir historial de git para "sacar" una llave que nunca estuvo ahí.

Auditoría real corrida sobre este repo (ver README.md, sección de troubleshooting/ auditoría, y el historial de esta conversación): `git log --all -p | grep "AIza[0-9A-Za-z_-]{35}"` → **0 coincidencias**. `git ls-files | grep "firebase-config.js"` → solo `firebase-config.example.js` (la plantilla).

`firestore.rules` **comentadas línea por línea**

```

rules_version = '2';                                     // L1: sintaxis v2 (obligatoria desde 2019)

service cloud.firestore {
  match /databases/{database}/documents {                // L4: aplica a toda la base de datos

    // games: catálogo de juegos, solo lectura para usuarios autenticados.
    // La escritura se hace desde el script de seed (Admin SDK), nunca desde el cliente.
    match /games/{gameId} {                               // L8: colección `games`, cualquier documento
      allow read: if request.auth != null;                // L9: cualquier usuario CON sesión puede leer el
      catálogo                                           // L10: NADIE escribe desde cliente – ni con
      allow write: if false;                               // El Admin SDK (seed_games.js) se salta
    }                                                    // por diseño (usa credenciales de
    las reglas                                           // pasa por este archivo). Evita que un
    servicio, no                                         // malicioso modifique
    usuario                                             // precios/dificultad/catálogo.

    // users/{uid}: perfil propio del usuario, y su subcolección de sesiones.
    match /users/{uid} {
      allow read, write: if request.auth != null          // L15: solo el DUEÑO del uid puede leer/escribir
      && request.auth.uid == uid;                          // su propio documento de perfil – nadie
    }                                                      // ni siquiera otro usuario autenticado.

    match /{document=**} {                                // L17: comodín recursivo – cualquier
    subcolección                                          // futura bajo users/{uid} hereda la MISMA
    allow read, write: if request.auth != null           // de propiedad, sin tener que declararla
    regla                                                // a mano.
    && request.auth.uid == uid;
    }
    match /sessions/{sessionId} {                         // L20: redundante con el comodín de arriba,
    pero                                                  // explícito a propósito – deja claro en el
    allow read, write: if request.auth != null           // archivo que las sesiones guardadas son
    && request.auth.uid == uid;                           // privadas por diseño, no un descuido.
    }

    match /verificacion/{docId} {                        // L24: el código de 2FA (sección 2) – mismo
    dueño,                                              // mismo criterio. Importante: esto NO
    allow read, write: if request.auth != null           // el propio usuario lea su código sin
    evita que                                           // "recibido" por otro canal – es la
    && request.auth.uid == uid;                           // ya documentada de no tener Cloud
    haberlo                                           // Functions
    }                                                  // (plan Spark) para enviarlo por correo
    limitación
    real.
  }

  // sessionState/{uid}: estado en vivo de la sesión, corazón del sync teléfono-TV.
  match /sessionState/{uid} {                            // L31: un documento por usuario, id = su
  propio uid                                           // L32: MISMA cuenta lee y escribe – el teléfono
  allow read, write: if request.auth != null           // escribe cada 2s, la TV (logueada con LA
  && request.auth.uid == uid;                           // cuenta) lee vía onSnapshot. Ningún
  MISMA                                              // puede ver el estado en vivo de otra
  }                                                  // persona.
  otro uid
  persona.
}
}

```

Patrón de fondo en todo el archivo: cero reglas usan `allow read, write: if true` en ninguna colección con datos de usuario. La única regla permisiva (`games`) es de solo lectura y para un catálogo sin datos personales.

4. Aviso de privacidad completo

Versión completa (para consulta legal/evaluación). La versión corta que se muestra dentro de las apps está en `docs/aviso_privacidad.md` y en `telefono_app/lib/ui/privacy_screen.dart` / `smart_pwa` (sección 4 de ese documento tiene el detalle de implementación).

AVISO DE PRIVACIDAD — MIND GAMES Responsable del tratamiento Juan Luis Mendoza Hernandez, matrícula 2023371106, proyecto académico desarrollado para la Universidad Tecnológica de Querétaro (UTEQ), materia Desarrollo para Dispositivos Inteligentes. Contacto: a través de la plataforma institucional de la UTEQ. **Datos personales que se recaban** Nombre, correo electrónico, contraseña (gestionada por Firebase Authentication, nunca almacenada en texto plano por esta aplicación), y — mientras usas una sesión de juego con el wearable conectado — datos de ritmo cardiaco, tiempo de sesión, movimientos y nivel de concentración. El ritmo cardiaco se trata como **dato sensible** (dato de salud) conforme al artículo 3, fracción VI de la LFPDPPP. **Finalidades** (a) Crear y autenticar tu cuenta; (b) sincronizar tu sesión de juego entre el reloj, el teléfono y la pantalla de TV en tiempo real; (c) mostrarte estadísticas de tus propias sesiones; (d) enviarte una alerta si tu tiempo de juego continuo supera 30 minutos, tu ritmo cardiaco supera 120 lpm, o tu nivel de concentración cae por debajo de 30%. Ninguna finalidad es publicitaria ni se comparten datos con terceros ajenos a la infraestructura técnica necesaria para operar la app (ver siguiente punto). **Transferencias** Tus datos se almacenan y procesan mediante **Google LLC / Firebase** (Firebase Authentication y Cloud Firestore), cuya infraestructura puede ubicarse en servidores dentro de **Estados Unidos**. Esta transferencia es necesaria para la prestación del servicio (LFPDPPP Art. 37, fracción II) y está sujeta a los mecanismos de protección de datos de Google (<https://cloud.google.com/terms/data-processing-terms>). No se realizan otras transferencias a terceros. **Derechos ARCO** Puedes ejercer tus derechos de **Acceso, Rectificación, Cancelación y Oposición** sobre tus datos personales en cualquier momento. Para Acceso y Rectificación de nombre/correo, puedes hacerlo directamente desde la app (edición de perfil). Para Cancelación (borrado de cuenta) u Oposición, contacta al responsable por el medio indicado arriba. **Procedimiento y plazo de respuesta** Al recibir una solicitud ARCO, el responsable confirmará su identidad, resolverá la solicitud en un plazo máximo de **20 días hábiles** (conforme al artículo 32 de la LFPDPPP), y en caso de proceder, la hará efectiva dentro de los **15 días hábiles** siguientes. Para solicitudes de cancelación de cuenta, el procedimiento técnico exacto está documentado en la sección 5 (Plan de retención) de este documento. **Cambios a este aviso** Cualquier cambio sustancial a este aviso se notificará dentro de la propia aplicación antes de que surta efecto.

5. Plan de retención

Dato: Perfil (<code>nombre</code> , <code>email</code>)	
Dónde	<code>users/{uid}</code>
Se conserva	Mientras la cuenta exista
Procedimiento técnico de eliminación	Borrado del documento Firestore + <code>FirebaseAuth.instance.currentUser.delete()</code> (equivalente a <code>deleteUser</code> de Admin SDK)

Dato: Sesiones guardadas	
Dónde	<code>users/{uid}/sessions/*</code>
Se conserva	Mientras la cuenta exista (histórico de estadísticas)
Procedimiento técnico de eliminación	Borrado en cascada: como no hay Cloud Functions (plan Spark) para borrado automático de subcolecciones, se hace con un batch delete manual antes de borrar el documento padre — Firestore NO borra subcolecciones solas al borrar el documento contenedor

Dato: Código de verificación 2FA	
Dónde	<code>users/{uid}/verificacion/actual</code>
Se conserva	5 minutos lógicos (campo <code>expiraEn</code>), pero el DOCUMENTO en sí no se autoelimina de Firestore — limitación conocida , requeriría un Cloud Function programado (plan Blaze) para purga real
Procedimiento técnico de eliminación	Se sobrescribe en cada nuevo login (mismo <code>docId</code> : <code>'actual'</code> , <code>set()</code> no <code>add()</code>) — nunca acumula histórico de códigos viejos, aunque el último sí persiste más allá de sus 5 minutos de validez lógica

Dato: Estado en vivo de sesión	
Dónde	<code>sessionState/{uid}</code>
Se conserva	Mientras la cuenta exista; se sobrescribe cada 2 s
Procedimiento técnico de eliminación	Borrado del documento único al eliminar la cuenta

Dato: Sesión de Firebase Auth (token)	
Dónde	Almacenamiento interno de Firebase Auth SDK (Android: interno de la app, no accesible directamente; PWA: IndexedDB del navegador)
Se conserva	Hasta <code>signOut()</code> explícito
Procedimiento técnico de eliminación	<code>AuthRepository.cerrarSesion()</code> (teléfono) / tecla "L" → <code>signOut(auth)</code> (PWA, <code>smart_pwa/js/app.js</code>) — limpia el token persistido

Dato: Caché del Service Worker (PWA)	
Dónde	Cache Storage del navegador, bajo la clave <code>CACHE_VERSION</code> (actualmente <code>mind-games-tv-v11</code>)
Se conserva	Hasta la siguiente activación de una versión nueva
Procedimiento técnico de eliminación	Automático: <code>sw.js</code> , evento <code>activate</code> — <code>caches.keys().then(...).filter(n => n !== CACHE_VERSION).map(caches.delete)</code> borra toda caché de versiones anteriores sin intervención del usuario

Dato: Métricas del wearable	
Dónde	No se guardan en el wearable — <code>SensorSimulator</code> las mantiene solo en memoria del proceso (<code>_sessionSeconds</code> , etc.), se pierden al cerrar la app
Se conserva	N/A (no hay persistencia)
Procedimiento técnico de eliminación	N/A

Nota de transparencia: este proyecto NO usa `shared_preferences` en `telefono_app` (se verificó que el paquete no está entre las dependencias) — la única persistencia local relevante es la interna del SDK de Firebase Auth (token de sesión), que ya se cubre en la fila correspondiente de la tabla.

6. Checklist de seguridad de la PWA

Control: CSP	
Estado	✓ Implementado
Detalle real	<code>smart_pwa/index.html</code> : <code>default-src 'self'; script-src 'self' https://apis.google.com; frame-src https://mind-games-ddi.firebaseio.com; connect-src 'self' https://firestore.googleapis.com https://identitytoolkit.googleapis.com https://securetoken.googleapis.com wss://firestore.googleapis.com; object-src 'none'; base-uri 'self'; frame-ancestors 'none';</code> — las dos excepciones a <code>'self'</code> (<code>apis.google.com</code> , <code>mind-games-ddi.firebaseio.com</code>) son mínimas y documentadas en el propio HTML: son requisito interno de Firebase Auth (<code>AuthEventManager</code>), encontrado en vivo durante las pruebas de esta evaluación.

Control: HTTPS	
Estado	⚠ Contexto seguro local
Detalle real	La demo corre en <code>http://localhost:8080</code> . Esto no es una vulnerabilidad : los navegadores tratan <code>localhost</code> (y <code>127.0.0.1</code>) como <i>contexto seguro</i> aunque no use TLS — por eso <code>Service Worker</code> , <code>BroadcastChannel</code> y el resto de APIs que exigen contexto seguro funcionan sin problema. Un despliegue real a un dominio público Sí requeriría HTTPS genuino (ej. Firebase Hosting lo da gratis y automático).

Control: SRI (Subresource Integrity)	
Estado	– No aplica, por diseño
Detalle real	Los SDK de Firebase (<code>js/vendor/firebase-app.js</code> , <code>firebase-auth.js</code> , <code>firebase-firestore.js</code>) se descargaron una vez y se sirven desde <code>'self'</code> , no desde un CDN externo — SRI (<code>integrity="sha384-..."</code>) solo tiene sentido para recursos cargados desde un origen que no controlas, para detectar si el CDN fue comprometido. Servir los archivos localmente es preferible a CDN+SRI aquí: (1) la CSP puede quedar 100% en <code>'self'</code> para <code>script-src</code> de los SDK (nunca hay que confiar en que un CDN de terceros no sea comprometido), (2) la app funciona offline sin depender de que el CDN esté disponible, (3) no hay una petición de red extra en el camino crítico del login.

Control: Validación de origen	
Estado	✓ Implementado
Detalle real	Ver sección 1 (<code>BroadcastChannel</code>) — único canal de mensajería entre contextos de la PWA, con <code>event.origin</code> validado.

Control: Reglas de acceso a datos	
Estado	✓ Implementado
Detalle real	Ver sección 3 (<code>firestore.rules</code>) — todo acceso a datos de usuario exige <code>request.auth.uid == uid</code> .